```
"""
AI Chat Application - LLM APIs &
Application Development (Module 3)
Features: LLM API connection (env
variables), chat history,
system/user/assistant
messages, custom instructions, parameter +
token control, structured (JSON)
responses, robust error handling with
retries, and API cost tracking.
"""
import json
import os
import time
from datetime import datetime

import streamlit as st
from dotenv import load_dotenv
from openai import (
    APIConnectionError,
    APIStatusError,
    APITimeoutError,
    AuthenticationError,
    OpenAI,
    RateLimitError,
)

load_dotenv()

st.set_page_config(page_title="AI Chat
```

```
Assistant", page_icon="🤖",
layout="centered")

# ----------------------------------------
---------------------- config
PROVIDERS = {
    "Groq": {
        "base_url":
"https://api.groq.com/openai/v1",
        "key_name": "GROQ_API_KEY",
        "models": ["llama-3.3-70b-
versatile", "llama-3.1-8b-instant"],
        "key_help": "Free key:
console.groq.com/keys",
    },
    "Google Gemini": {
        "base_url":
"https://generativelanguage.googleapis.com/
v1beta/openai/",
        "key_name": "GEMINI_API_KEY",
        "models": ["gemini-3.8-flash"],
        "key_help": "Free key:
aistudio.google.com/apikey",
    },
}

PRESETS = {
    "Helpful assistant": "You are a
helpful, concise AI assistant. Explain
things clearly.",
    "Python tutor": "You are a patient
Python tutor. Explain step by step with
short code examples.",
    "Interview coach": "You are an
```

```
interview coach for fresher engineers. Ask
one question at a time and give feedback.",
    "Tamil + English helper": "You are a
friendly assistant. Reply in simple
Tanglish (Tamil written in English letters)
unless asked otherwise.",
    "Custom": "",
}

STRUCTURED_RULES = (
    "Respond ONLY with a valid JSON object
with exactly these keys: "
    '"answer" (string, markdown allowed),
"summary" (one-sentence string), '
    '"key_points" (array of up to 4 short
strings), '
    '"confidence" (one of "high", "medium",
"low"). No text outside the JSON.'
)


def get_secret(name: str) -> str:
    """Read a key from Streamlit secrets or
environment variables."""
    try:
        if name in st.secrets:
            return st.secrets[name]
    except Exception:
        pass
    return os.getenv(name, "")


def approx_tokens(text: str) -> int:
    return max(1, len(text) // 4)
```

```
# ----------------------------------------
---------------------- LLM call
def call_llm(client, model, messages,
temperature, max_tokens, json_mode,
retries=3):
    """Call the chat API with retry +
friendly errors. Returns (text, usage) or
raises RuntimeError."""
    kwargs = dict(model=model,
messages=messages, temperature=temperature,
max_tokens=max_tokens)
    if json_mode:
        kwargs["response_format"] =
{"type": "json_object"}

    for attempt in range(1, retries + 1):
        try:
            resp =
client.chat.completions.create(**kwargs,
timeout=60)
            text =
resp.choices[0].message.content or ""
            usage = resp.usage
            return text, {
                "prompt": getattr(usage,
"prompt_tokens", 0) or 0,
                "completion":
getattr(usage, "completion_tokens", 0) or
0,
            }
        except AuthenticationError:
            raise RuntimeError("Invalid API
```

```
key. Please check your key in the sidebar / secrets.")
        except RateLimitError:
            if attempt == retries:
                raise RuntimeError("Rate limit reached. Please wait a minute and try again.")
            time.sleep(2 ** attempt)
        except (APIConnectionError, APITimeoutError):
            if attempt == retries:
                raise RuntimeError("Network problem / timeout. Check your internet and retry.")
            time.sleep(2 ** attempt)
        except APIStatusError as e:
            if e.status_code >= 500 and attempt < retries:
                time.sleep(2 ** attempt)
                continue
            raise RuntimeError(f"API error {e.status_code}: {getattr(e, 'message', str(e))}")
        except Exception as e:  # last-resort guard
            raise RuntimeError(f"Unexpected error: {e}")


def parse_structured(text: str):
    try:
        data = json.loads(text)
        if isinstance(data, dict) and "answer" in data:
```

```
            return data
    except json.JSONDecodeError:
        pass
    return None


# ----------------------------------------
---------------------- state
if "messages" not in st.session_state:
    st.session_state.messages = []  #
{"role","content","meta"}
if "stats" not in st.session_state:
    st.session_state.stats = {"requests":
0, "prompt": 0, "completion": 0, "cost":
0.0}

# ----------------------------------------
---------------------- sidebar
with st.sidebar:
    st.header("⚙️ Settings")
    provider_name =
st.selectbox("Provider", list(PROVIDERS))
    provider = PROVIDERS[provider_name]
    model = st.selectbox("Model",
provider["models"])

    api_key =
get_secret(provider["key_name"])
    if not api_key:
        api_key = st.text_input("API key",
type="password", help=provider["key_help"])
        st.caption(provider["key_help"])
    else:
        st.success("API key loaded from
```

```
environment ✅")

    st.subheader("🎯 Custom instructions")
    preset = st.selectbox("Preset",
list(PRESETS))
    instructions = st.text_area(
        "System prompt",
        value=PRESETS[preset],
        height=110,
        key=f"instr_{preset}",
        placeholder="Tell the AI how to
behave...",
    )

    st.subheader("🎛️ Model parameters")
    temperature = st.slider("Temperature",
0.0, 1.5, 0.7, 0.1)
    max_tokens = st.slider("Max response
tokens", 100, 2000, 600, 50)
    history_limit = st.slider("Messages
kept in context", 2, 20, 8, 2,
                              help="Older
messages are dropped to save tokens.")
    structured = st.toggle("Structured
(JSON) responses", value=False)

    st.subheader("💰 Cost tracking")
    price_in = st.number_input("Price / 1M
input tokens ($)", 0.0, 100.0, 0.0, 0.05)
    price_out = st.number_input("Price / 1M
output tokens ($)", 0.0, 100.0, 0.0, 0.05)

    s = st.session_state.stats
    st.metric("Requests", s["requests"])
```

```
    c1, c2 = st.columns(2)
    c1.metric("Input tok", s["prompt"])
    c2.metric("Output tok",
s["completion"])
    st.metric("Estimated cost",
f"${s['cost']:.5f}")

    col_a, col_b = st.columns(2)
    if col_a.button("🗑️ Clear chat",
use_container_width=True):
        st.session_state.messages = []
        st.session_state.stats =
{"requests": 0, "prompt": 0, "completion":
0, "cost": 0.0}
        st.rerun()
    col_b.download_button(
        "⬇️ Export",

data=json.dumps(st.session_state.messages,
indent=2, ensure_ascii=False),

file_name=f"chat_{datetime.now():%Y%m%d_%H%
M}.json",
        mime="application/json",
        use_container_width=True,
    )

# ----------------------------------------
---------------------- main UI
st.title("🤖 AI Chat Assistant")
st.caption(f"Powered by {provider_name} ·
{model}")
```

```
def render_assistant(msg):
    meta = msg.get("meta") or {}
    st.markdown(msg["content"])
    structured_data =
meta.get("structured")
    if structured_data:
        with st.expander("📋 Structured
details"):
            st.write(f"**Summary:**
{structured_data.get('summary', '-')}")
            for p in
structured_data.get("key_points", []) or
[]:
                st.write(f"- {p}")
            st.write(f"**Confidence:**
{structured_data.get('confidence', '-')}")
    if meta.get("usage"):
        u = meta["usage"]
        st.caption(f"🔢 {u['prompt']} in +
{u['completion']} out tokens")


if not st.session_state.messages:
    st.info("👋 Hi! Ask me anything. Change
the custom instructions in the sidebar to
change my behaviour.")

for m in st.session_state.messages:
    with st.chat_message(m["role"]):
        if m["role"] == "assistant":
            render_assistant(m)
        else:
            st.markdown(m["content"])
```

```
user_input = st.chat_input("Type your
message...")

if user_input:
    if not api_key:
        st.error("Please add your API key
in the sidebar first.")
        st.stop()
    if len(user_input) > 4000:
        st.warning("Message too long (max
4000 characters).")
        st.stop()


st.session_state.messages.append({"role":
"user", "content": user_input})
    with st.chat_message("user"):
        st.markdown(user_input)

    # Build API messages: system + trimmed
history
    system_prompt = (instructions or
PRESETS["Helpful assistant"]).strip()
    if structured:
        system_prompt += "\n\n" +
STRUCTURED_RULES
    history = [{"role": m["role"],
"content": m["content"]} for m in
st.session_state.messages]
    api_messages = [{"role": "system",
"content": system_prompt}] + history[-
history_limit:]

    with st.chat_message("assistant"):
```

```
        try:
            client =
OpenAI(api_key=api_key,
base_url=provider["base_url"])
            with st.spinner("Thinking..."):
                raw, usage =
call_llm(client, model, api_messages,
temperature, max_tokens, structured)

            meta = {"usage": usage}
            content = raw
            if structured:
                data =
parse_structured(raw)
                if data:
                    content =
str(data["answer"])
                    meta["structured"] =
data
                else:
                    st.warning("Model did
not return valid JSON, showing plain
text.")

            reply = {"role": "assistant",
"content": content, "meta": meta}
            render_assistant(reply)

st.session_state.messages.append(reply)

            st_ = st.session_state.stats
            st_["requests"] += 1
            st_["prompt"] +=
usage["prompt"]
```

```
            st_["completion"] +=
usage["completion"]
            st_["cost"] += (usage["prompt"]
* price_in + usage["completion"] *
price_out) / 1_000_000
            st.rerun()
        except RuntimeError as err:
            st.error(f"⚠️ {err}")

st.session_state.messages.pop()  # drop
unanswered user message
```